```
{
 "nbformat": 4,
 "nbformat_minor": 0,
 "metadata": {
  "colab": {
   "provenance": []
  },
  "kernelspec": {
   "name": "python3",
   "display_name": "Python 3"
  },
  "language_info": {
   "name": "python"
  }
 },
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {
    "id": "kqB21QOgMg-G"
   },
   "source": [
    "Importing the Dependencies"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {
    "id": "rALl06-oHusw"
   },
   "source": [
    "import numpy as np\n",
    "import pandas as pd\n",
```

```
    "from sklearn.model_selection import train_test_split\n",
    "from sklearn.feature_extraction.text import TfidfVectorizer\n",
    "from sklearn.linear_model import LogisticRegression\n",
    "from sklearn.metrics import accuracy_score"
   ],
   "execution_count": null,
   "outputs": []
  },
  {
   "cell_type": "markdown",
   "metadata": {
    "id": "YyKe9o2ONeFv"
   },
   "source": [
    "Data Collection & Pre-Processing"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {
    "id": "CpStHH8KNcYB"
   },
   "source": [
    "# loading the data from csv file to a pandas Dataframe\n",
    "raw_mail_data = pd.read_csv('/content/mail_data.csv')"
   ],
   "execution_count": null,
   "outputs": []
  },
  {
   "cell_type": "code",
   "metadata": {
    "colab": {
```

```
    "base_uri": "https://localhost:8080/"
   },
   "id": "pdn-7VE2NxsZ",
   "outputId": "28c19d96-23a2-43c0-86ad-5c1aee7f1b58"
  },
  "source": [
   "print(raw_mail_data)"
  ],
  "execution_count": null,
  "outputs": [
   {
    "output_type": "stream",
    "name": "stdout",
    "text": [
     "    Category                                Message\n",
     "0       ham  Go until jurong point, crazy.. Available only ...\n",
     "1       ham                  Ok lar... Joking wif u oni...\n",
     "2      spam  Free entry in 2 a wkly comp to win FA Cup fina...\n",
     "3       ham  U dun say so early hor... U c already then say...\n",
     "4       ham  Nah I don't think he goes to usf, he lives aro...\n",
     "...     ...                                   ...\n",
     "5567    spam  This is the 2nd time we have tried 2 contact u...\n",
     "5568     ham            Will ü b going to esplanade fr home?\n",
     "5569     ham  Pity, * was in mood for that. So...any other s...\n",
     "5570     ham  The guy did some bitching but I acted like i'd...\n",
     "5571     ham                    Rofl. Its true to its name\n",
     "\n",
     "[5572 rows x 2 columns]\n"
    ]
   }
  ]
 },
 {
```

```
 "cell_type": "code",
 "metadata": {
  "id": "yhakjlE1N011"
 },
 "source": [
  "# replace the null values with a null string\n",
  "mail_data = raw_mail_data.where((pd.notnull(raw_mail_data)),'')"
 ],
 "execution_count": null,
 "outputs": []
},
{
 "cell_type": "code",
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/",
   "height": 202
  },
  "id": "SJey6H-SOWeK",
  "outputId": "af1b0dfd-2ff9-4af9-cfcd-d0c177dd6ab9"
 },
 "source": [
  "# printing the first 5 rows of the dataframe\n",
  "mail_data.head()"
 ],
 "execution_count": null,
 "outputs": [
  {
   "output_type": "execute_result",
   "data": {
    "text/html": [
     "<div>\n",
     "<style scoped>\n",
```

```
"    .dataframe tbody tr th:only-of-type {\n",
"        vertical-align: middle;\n",
"    }\n",
"\n",
"    .dataframe tbody tr th {\n",
"        vertical-align: top;\n",
"    }\n",
"\n",
"    .dataframe thead th {\n",
"        text-align: right;\n",
"    }\n",
"</style>\n",
"<table border=\"1\" class=\"dataframe\">\n",
"  <thead>\n",
"    <tr style=\"text-align: right;\">\n",
"      <th></th>\n",
"      <th>Category</th>\n",
"      <th>Message</th>\n",
"    </tr>\n",
"  </thead>\n",
"  <tbody>\n",
"    <tr>\n",
"      <th>0</th>\n",
"      <td>ham</td>\n",
"      <td>Go until jurong point, crazy.. Available only ...</td>\n",
"    </tr>\n",
"    <tr>\n",
"      <th>1</th>\n",
"      <td>ham</td>\n",
"      <td>Ok lar... Joking wif u oni...</td>\n",
"    </tr>\n",
"    <tr>\n",
"      <th>2</th>\n",
```

```
      "    <td>spam</td>\n",
      "    <td>Free entry in 2 a wkly comp to win FA Cup fina...</td>\n",
      "   </tr>\n",
      "   <tr>\n",
      "    <th>3</th>\n",
      "    <td>ham</td>\n",
      "    <td>U dun say so early hor... U c already then say...</td>\n",
      "   </tr>\n",
      "   <tr>\n",
      "    <th>4</th>\n",
      "    <td>ham</td>\n",
      "    <td>Nah I don't think he goes to usf, he lives aro...</td>\n",
      "   </tr>\n",
      "  </tbody>\n",
      "</table>\n",
      "</div>"
     ],
     "text/plain": [
      "  Category                                            Message\n",
      "0      ham  Go until jurong point, crazy.. Available only ...\n",
      "1      ham                      Ok lar... Joking wif u oni...\n",
      "2     spam  Free entry in 2 a wkly comp to win FA Cup fina...\n",
      "3      ham  U dun say so early hor... U c already then say...\n",
      "4      ham  Nah I don't think he goes to usf, he lives aro..."
     ]
    },
    "metadata": {},
    "execution_count": 5
   }
  ]
 },
 {
  "cell_type": "code",
```

```
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/"
  },
  "id": "lbK82N2gOdar",
  "outputId": "4d1840a1-22b5-468f-d4d0-a4528ef4313c"
 },
 "source": [
  "# checking the number of rows and columns in the dataframe\n",
  "mail_data.shape"
 ],
 "execution_count": null,
 "outputs": [
  {
   "output_type": "execute_result",
   "data": {
    "text/plain": [
     "(5572, 2)"
    ]
   },
   "metadata": {},
   "execution_count": 6
  }
 ]
},
{
 "cell_type": "markdown",
 "metadata": {
  "id": "vhR4U3ATPBdk"
 },
 "source": [
  "Label Encoding"
 ]
```

```
},
{
 "cell_type": "code",
 "metadata": {
  "id": "9EW7QSgeOt4p"
 },
 "source": [
  "# label spam mail as 0;  ham mail as 1;\n",
  "\n",
  "mail_data.loc[mail_data['Category'] == 'spam', 'Category',] = 0\n",
  "mail_data.loc[mail_data['Category'] == 'ham', 'Category',] = 1"
 ],
 "execution_count": null,
 "outputs": []
},
{
 "cell_type": "markdown",
 "metadata": {
  "id": "uxZK1fWwPwll"
 },
 "source": [
  "spam  -  0\n",
  "\n",
  "ham  -  1"
 ]
},
{
 "cell_type": "code",
 "metadata": {
  "id": "t8Rt-FaNPtPE"
 },
 "source": [
  "# separating the data as texts and label\n",
```

```
    "\n",
    "X = mail_data['Message']\n",
    "\n",
    "Y = mail_data['Category']"
   ],
   "execution_count": null,
   "outputs": []
  },
  {
   "cell_type": "code",
   "metadata": {
    "colab": {
     "base_uri": "https://localhost:8080/"
    },
    "id": "QnQeUBGtQPP7",
    "outputId": "a2640f4b-2a1d-4742-9742-3ecbb6017668"
   },
   "source": [
    "print(X)"
   ],
   "execution_count": null,
   "outputs": [
    {
     "output_type": "stream",
     "name": "stdout",
     "text": [
      "0     Go until jurong point, crazy.. Available only ...\n",
      "1                     Ok lar... Joking wif u oni...\n",
      "2     Free entry in 2 a wkly comp to win FA Cup fina...\n",
      "3     U dun say so early hor... U c already then say...\n",
      "4     Nah I don't think he goes to usf, he lives aro...\n",
      "                        ...                   \n",
      "5567    This is the 2nd time we have tried 2 contact u...\n",
```

```
      "5568              Will ü b going to esplanade fr home?\n",
      "5569   Pity, * was in mood for that. So...any other s...\n",
      "5570   The guy did some bitching but I acted like i'd...\n",
      "5571                     Rofl. Its true to its name\n",
      "Name: Message, Length: 5572, dtype: object\n"
     ]
    }
   ]
  },
  {
   "cell_type": "code",
   "metadata": {
    "colab": {
     "base_uri": "https://localhost:8080/"
    },
    "id": "cuWDNy5KQQjY",
    "outputId": "1a0a109b-d63a-4cf0-fe4e-b486f1d3d623"
   },
   "source": [
    "print(Y)"
   ],
   "execution_count": null,
   "outputs": [
    {
     "output_type": "stream",
     "name": "stdout",
     "text": [
      "0     1\n",
      "1     1\n",
      "2     0\n",
      "3     1\n",
      "4     1\n",
      "      ..\n",
```

```
        "5567   0\n",
        "5568   1\n",
        "5569   1\n",
        "5570   1\n",
        "5571   1\n",
        "Name: Category, Length: 5572, dtype: object\n"
       ]
      }
     ]
    },
    {
     "cell_type": "markdown",
     "metadata": {
      "id": "jvHyqdH8QZPH"
     },
     "source": [
      "Splitting the data into training data & test data"
     ]
    },
    {
     "cell_type": "code",
     "metadata": {
      "id": "RO2GmbSNQSQH"
     },
     "source": [
      "X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random
_state=3)"
     ],
     "execution_count": null,
     "outputs": []
    },
    {
     "cell_type": "code",
```

```
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/"
  },
  "id": "tS2c7A4NRa46",
  "outputId": "5d44247f-65d0-457d-8a94-0fd8b45a3b72"
 },
 "source": [
  "print(X.shape)\n",
  "print(X_train.shape)\n",
  "print(X_test.shape)"
 ],
 "execution_count": null,
 "outputs": [
  {
   "output_type": "stream",
   "name": "stdout",
   "text": [
    "(5572,)\n",
    "(4457,)\n",
    "(1115,)\n"
   ]
  }
 ]
},
{
 "cell_type": "markdown",
 "metadata": {
  "id": "wYQpiACGSBYM"
 },
 "source": [
  "Feature Extraction"
 ]
```

```
  },
  {
   "cell_type": "code",
   "metadata": {
    "id": "nLs847nSRibm"
   },
   "source": [
    "# transform the text data to feature vectors that can be used as input to the L
ogistic regression\n",
    "\n",
    "feature_extraction = TfidfVectorizer(min_df = 1, stop_words='english', lower
case='True')\n",
    "\n",
    "X_train_features = feature_extraction.fit_transform(X_train)\n",
    "X_test_features = feature_extraction.transform(X_test)\n",
    "\n",
    "# convert Y_train and Y_test values as integers\n",
    "\n",
    "Y_train = Y_train.astype('int')\n",
    "Y_test = Y_test.astype('int')"
   ],
   "execution_count": null,
   "outputs": []
  },
  {
   "cell_type": "code",
   "metadata": {
    "id": "dBMAcw9RUkUY"
   },
   "source": [
    "print(X_train)"
   ],
   "execution_count": null,
```

```
  "outputs": []
 },
 {
  "cell_type": "code",
  "metadata": {
   "id": "1NFuGogZUpt0"
  },
  "source": [
   "print(X_train_features)"
  ],
  "execution_count": null,
  "outputs": []
 },
 {
  "cell_type": "markdown",
  "metadata": {
   "id": "q86FvELbU_SV"
  },
  "source": [
   "Training the Model"
  ]
 },
 {
  "cell_type": "markdown",
  "metadata": {
   "id": "hV6BAIZQVBbo"
  },
  "source": [
   "Logistic Regression"
  ]
 },
 {
  "cell_type": "code",
```

```
   "metadata": {
    "id": "1JeAOwzpUv0V"
   },
   "source": [
    "model = LogisticRegression()"
   ],
   "execution_count": null,
   "outputs": []
  },
  {
   "cell_type": "code",
   "metadata": {
    "colab": {
     "base_uri": "https://localhost:8080/"
    },
    "id": "gWGRHWAPVI_z",
    "outputId": "1c5e15dd-0e07-4871-c4fa-b908ee400b55"
   },
   "source": [
    "# training the Logistic Regression model with the training data\n",
    "model.fit(X_train_features, Y_train)"
   ],
   "execution_count": null,
   "outputs": [
    {
     "output_type": "execute_result",
     "data": {
      "text/plain": [
       "LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=
True,\n",
       "                   intercept_scaling=1, l1_ratio=None, max_iter=100,\n",
       "                   multi_class='auto', n_jobs=None, penalty='l2',\n",
       "                   random_state=None, solver='lbfgs', tol=0.0001, verbose=0,\n",
```

```
          "                warm_start=False)"
        ]
       },
       "metadata": {},
       "execution_count": 18
      }
     ]
    },
    {
     "cell_type": "markdown",
     "metadata": {
      "id": "wZ01fa8dVeL5"
     },
     "source": [
      "Evaluating the trained model"
     ]
    },
    {
     "cell_type": "code",
     "metadata": {
      "id": "ExiF2kKxVYtC"
     },
     "source": [
      "# prediction on training data\n",
      "\n",
      "prediction_on_training_data = model.predict(X_train_features)\n",
      "accuracy_on_training_data = accuracy_score(Y_train, prediction_on_trainin
g_data)"
     ],
     "execution_count": null,
     "outputs": []
    },
    {
```

```
  "cell_type": "code",
  "metadata": {
   "colab": {
    "base_uri": "https://localhost:8080/"
   },
   "id": "o7t4Dl5UWCkB",
   "outputId": "49fafbb0-0e7f-40c7-9ab7-4aea165731ee"
  },
  "source": [
   "print('Accuracy on training data : ', accuracy_on_training_data)"
  ],
  "execution_count": null,
  "outputs": [
   {
    "output_type": "stream",
    "name": "stdout",
    "text": [
     "Accuracy on training data :  0.9670181736594121\n"
    ]
   }
  ]
 },
 {
  "cell_type": "code",
  "metadata": {
   "id": "cTin5rXTWKg3"
  },
  "source": [
   "# prediction on test data\n",
   "\n",
   "prediction_on_test_data = model.predict(X_test_features)\n",
   "accuracy_on_test_data = accuracy_score(Y_test, prediction_on_test_data)
"
```

```
 ],
 "execution_count": null,
 "outputs": []
},
{
 "cell_type": "code",
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/"
  },
  "id": "4gvoMK4OWnJY",
  "outputId": "7bf56da4-1987-4828-ea00-95c30fb083d1"
 },
 "source": [
  "print('Accuracy on test data : ', accuracy_on_test_data)"
 ],
 "execution_count": null,
 "outputs": [
  {
   "output_type": "stream",
   "name": "stdout",
   "text": [
    "Accuracy on test data :  0.9659192825112107\n"
   ]
  }
 ]
},
{
 "cell_type": "markdown",
 "metadata": {
  "id": "bXdOKxYAXaHC"
 },
 "source": [
```

```
    "Building a Predictive System"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {
    "colab": {
     "base_uri": "https://localhost:8080/"
    },
    "id": "h60z1__mWqI6",
    "outputId": "3aac53f3-13f2-4afb-e9f2-75d337cbcd44"
   },
   "source": [
    "input_mail = [\"I've been searching for the right words to thank you for this br eather. I promise i wont take your help for granted and will fulfil my promise. You h ave been wonderful and a blessing at all times\"]\n",
    "\n",
    "# convert text to feature vectors\n",
    "input_data_features = feature_extraction.transform(input_mail)\n",
    "\n",
    "# making prediction\n",
    "\n",
    "prediction = model.predict(input_data_features)\n",
    "print(prediction)\n",
    "\n",
    "\n",
    "if (prediction[0]==1):\n",
    "  print('Ham mail')\n",
    "\n",
    "else:\n",
    "  print('Spam mail')"
   ],
   "execution_count": null,
```

```
    "outputs": [
     {
      "output_type": "stream",
      "name": "stdout",
      "text": [
       "[1]\n",
       "Ham mail\n"
      ]
     }
    ]
   },
   {
    "cell_type": "code",
    "metadata": {
     "id": "v_LqbM_ZYwS1"
    },
    "source": [],
    "execution_count": null,
    "outputs": []
   }
  ]
}
```